

P1 — Backend & Database Lead

RoadAssist Bharat · 87 days across 3 reviews Owns: server services · PostgreSQL/PostGIS schema · all API contracts · authentication & RBAC · booking and dispatch engine · emergency core · payments · sync server. Backs up **P3**.

Your three standing rules 1. The API specification is generated from code and published on every merge. It is the single source of truth. 2. Nobody is ever blocked waiting on a contract from you. Specification and mock first, implementation second. 3. Every database migration is reviewed by a second person and has a tested rollback.

REVIEW 1 — Foundation · Weeks 1–5 · 25 days

Week	Task	Days
1	Integration feasibility: VAHAN/SARATHI, ERSS 112, UPI/NPCI, FASTag. Written go/no-go and a fallback for each.	2
1–2	Candidate entity list (~55 entities with cardinalities) from mechanic and fleet interviews	2
2	Backend backlog (~70 stories with acceptance criteria) + estimates; monorepo scaffold with CI	4
3	C4 architecture diagrams + 15 ADRs ; event catalogue; API style guide	5
3–4	60-table schema across 13 modules; ER diagram; index plan with query evidence	4
4	Audit log triggers; soft delete; optimistic-locking version columns; partitioning; seed generator (100k rows)	2
5	Authentication : OTP flow, JWT + rotating refresh with reuse detection, device binding	3
5	RBAC policy engine (7 roles × 45 permissions) + consent ledger (DPDP-compliant)	3

Deliverables

15 ADRs · C4 diagram set · API style guide · 60-table schema with migrations · ER diagram · seed generator · auth service · RBAC engine · consent ledger · TypeScript auth SDK.

Your Review 1 demo — 6 minutes

Run migrations on an empty database → seed 100,000 rows → run a geospatial "nearest mechanic" query showing **under 50 ms with the index and over a second without**. Update a record and show the audit trigger capturing before/after. Then log in by OTP on a real phone, steal the refresh token, replay it — and watch the entire session family get revoked and the user notified.

Passed when

Migrations roll back cleanly twice · every table has PK, timestamps, soft delete, version and an owner · zero sequential scans on the top-20 queries · OTP brute force blocked at 5 attempts per 15 minutes · 20 RBAC tests pass including 6 negative access-control cases.

REVIEW 2 — Core Product · Weeks 6–12 · 33 days

Week	Task	Days
6–8	140 API endpoints across 12 modules — users, vehicles, bookings, dispatch, mechanics, pricing, payments, notifications, fleet, reviews, support, webhooks	15
6–7	<i>(within the above)</i> Booking state machine with guarded transitions; dispatch scoring and offer ladder; pricing engine with transparent breakdown	—
8	<i>(within the above)</i> UPI payments, refunds, subscriptions; transactional outbox for reliable events	—
9	AI client wrapper: 800 ms timeout, circuit breaker, fallback registry , prediction logging	3
9	Feature pipeline into the feature store for P3	—
10	Composite BFF endpoints so the home screen is one request, not six (this matters enormously on 2G)	3
11–12	Sync server : cursor-based change feed, per-field conflict resolution, tombstones, batch replay	6

Week	Task	Days
12	Geofencing, live tracking channel, highway kilometre-marker resolution	3
12	Diagnostics service + DTC knowledge base (~5,000 codes) + mechanic pre-brief	3

Deliverables

12 service modules · 140 endpoints · OpenAPI 3.1 specification · generated TypeScript client · contract tests · booking state machine · sync server · **conflict resolution matrix** · geo services · diagnostics service.

Your Review 2 demo — 8 minutes

Scripted end to end: user requests help → 3 mechanics offered → one accepts → status moves to completion → invoice → UPI payment → review → partner webhook fires. **Then break it deliberately:** cancel mid-flight, time out a mechanic, fail a payment. Every path handled. Finally, kill the AI service completely and run the whole booking again on fallbacks.

Passed when

100% of the v1 API implemented · every write endpoint idempotent under replay · the state machine rejects every illegal transition · booking still completes with AI switched off · 30-scenario conflict suite passes.

REVIEW 3 — Complete System · Weeks 13–18 · 29 days

Week	Task	Days
13–14	Emergency engine — separate deployable, own database pool, own quota, spread across 3 zones	6
13	(within) Escalation ladder: contacts → responders → ambulance → ERSS 112 handoff with human confirmation	—
14	(within) Break-glass medical access with full audit; degraded SMS path sharing zero dependencies with the main platform	—
15	Government service: k-anonymity ≥ 10 enforced at the query layer , jurisdiction isolation, officer audit log	3
15	CDC pipeline Postgres → Kafka → ClickHouse with PII stripping	3
16	Property-based tests (state machine, pricing) + mutation testing; fix load and security findings	5
16	Verify security fixes from P4's penetration test	2
17	Online migrations (no locking), health probes, graceful shutdown, service runbooks	3
17–18	Query tuning from live statistics, caching layer, payload reduction, connection pooling	4
18	Scaling plan to 10M users with named bottlenecks; technical debt register	3

Deliverables

Emergency service (isolated) · 112 handoff adapter · break-glass audit · degraded SMS path · government service with anonymisation · CDC pipeline · property + mutation tests · service runbooks · scaling plan · debt register.

Your Review 3 demo — 7 minutes

Simulate a crash on a real device. Countdown → contacts get an SMS with a live location link → nearest responder notified → incident room opens, **under 10 seconds**. Then scale the entire rest of the platform to zero replicas and do it again — it still works. Finally, play attacker against the government API: try to identify one specific citizen, fail every time, and show the audit log that flagged you.

Passed when

Crash to responder under 10 seconds · emergency survives total platform failure · AI cannot auto-dispatch (enforced in code) · re-identification test fails on all 20 attack strategies · deploy a schema change to a 10-million-row table with zero downtime.

Your Key Design Decisions (defend these to the panel)

Decision	Why
Modular monolith, not microservices	Four people cannot operate 15 independently deployed services. Module boundaries are strict and split-ready. Emergency is the one genuine separate deployable — it needs a different reliability class.
Emergency deployed separately	Life safety needs its own failure domain. A bug in bookings must never be able to kill an SOS.
Server always wins on booking state	An offline client can never force an illegal state. State machine integrity is not negotiable.
AI is never on the correctness path	Every AI call has a deterministic fallback. Models advise; rules decide.
Money in integer paise, never floats	Floating-point arithmetic on currency produces reconciliation failures that are very expensive to find later.
Conflict matrix written before coding	The top technical risk in the project. Field-level rules are decided and reviewed before the first line of sync code.